Legged Robots Practical

10.10.2023

Plan

Single Leg

- **W1:** Introduction
- **W2:** Derivation of double pendulum's kinematics and dynamics
- **W3:** Jacobian (Cartesian PD + Force Control)
- **W4:** Inverse Kinematics (compare with force control)
- **W5:** Single-leg hopping

Biped

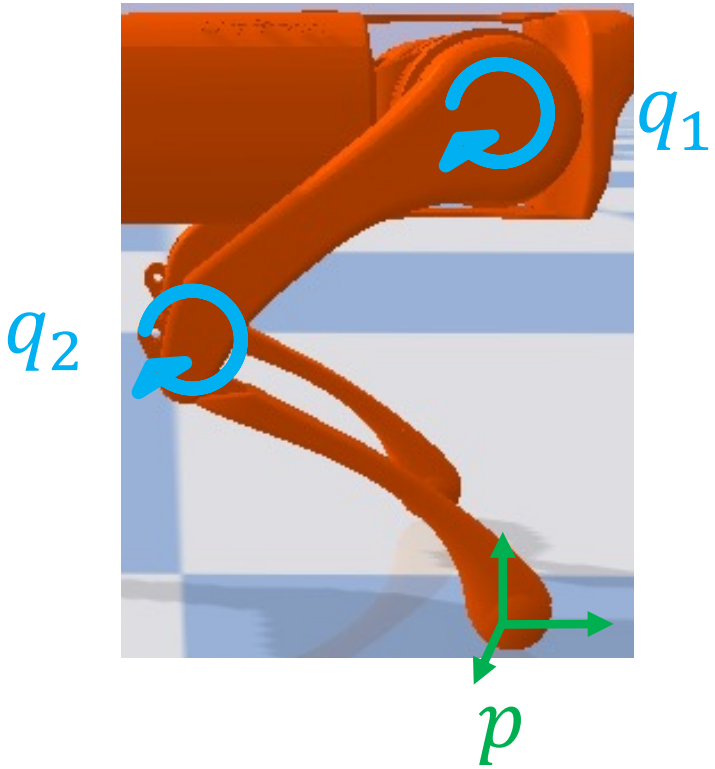
- **W6-8:** Biped Walking with Divergent Component of Motion (BD Atlas) **[Mini-Report]**

Quadruped

- **W9-10:** Quadruped CPG Trot
- **W11-14:** Quadruped Locomotion Project (CPGs, Deep RL) **[Report - Quadruped]**

Inverse Kinematics

Joint angles ←→ Cartesian space



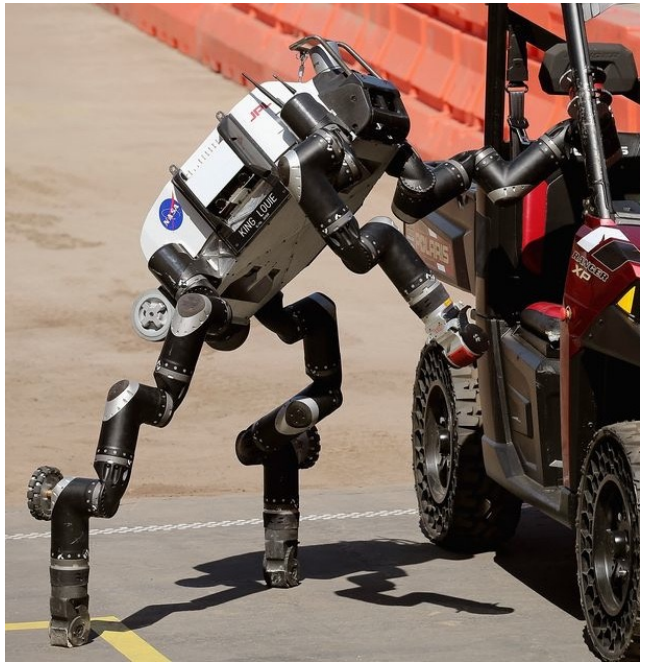
$p = f(q)$ Forward kinematics

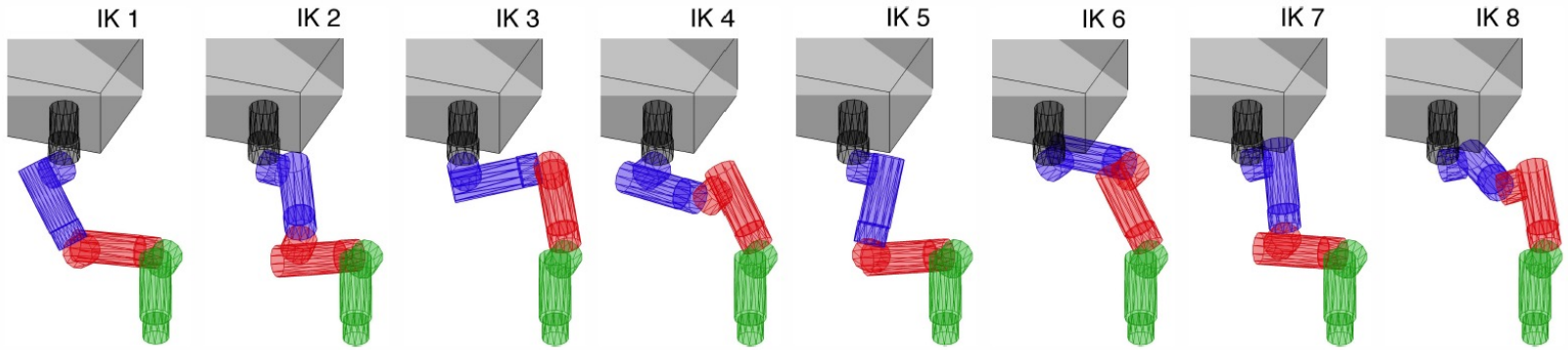
$q = f^{-1}(p)$ Inverse kinematics

$\dot{p} = v = J(q)\dot{q}$ Foot linear velocity

$\tau = J^T(q)F$ Map desired end effector force to torques

JPL's RoboSimian: Inverse Kinematics



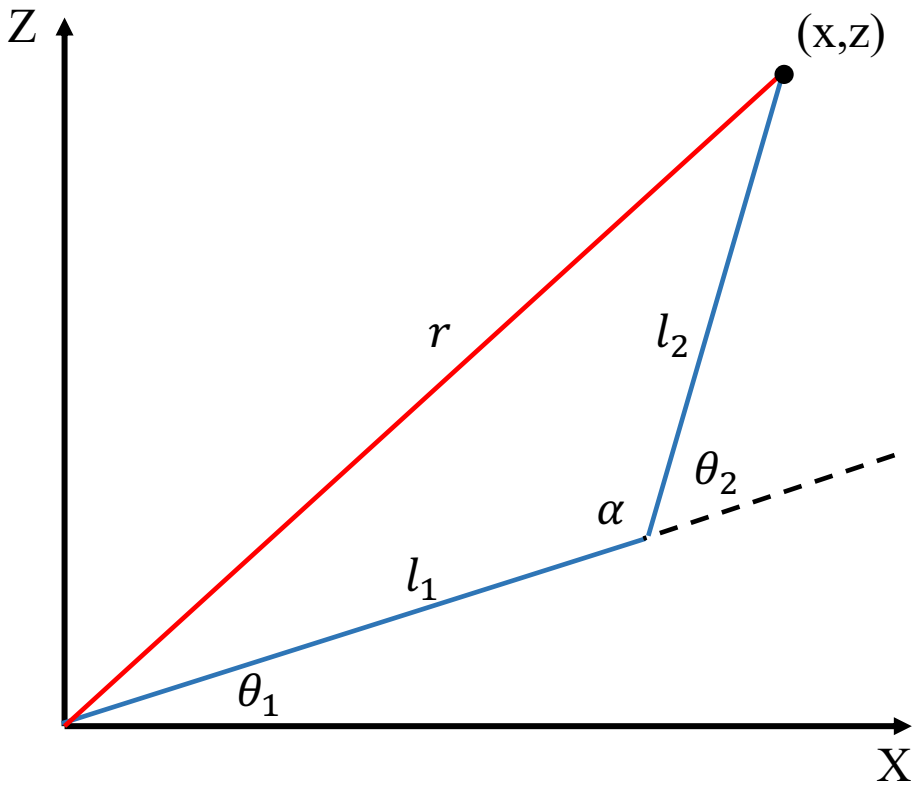


- Four identical limbs, each with 7 degrees of freedom (DOFs)
- 8 Inverse Kinematics "Families"
  - 3 "elbows" → 2^3 choices
- How should we choose between these?

Inverse Kinematics Today

1. Analytical
2. Iterative (numerical)

Analytical Inverse Kinematics



- Recall the cosine rule:

$$r^2 = l_1^2 + l_2^2 - 2l_1 l_2 \cos\alpha$$

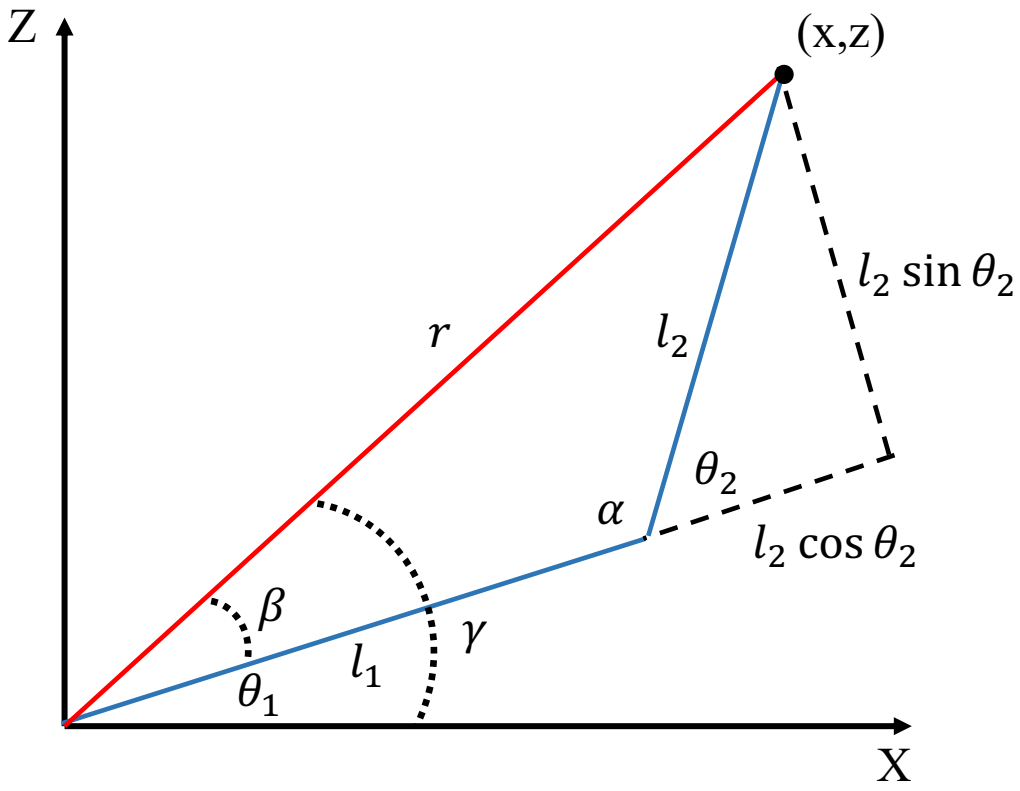
- Therefore:

$$\cos\alpha = \frac{l_1^2 + l_2^2 - r^2}{2l_1 l_2} = \frac{l_1^2 + l_2^2 - x^2 - z^2}{2l_1 l_2}$$

- Since $\theta_2 = \pi - \alpha$,

$$\cos\theta_2 = -\cos\alpha = \frac{r^2 - l_1^2 - l_2^2}{2l_1 l_2} = \frac{x^2 + z^2 - l_1^2 - l_2^2}{2l_1 l_2}$$

Analytical Inverse Kinematics



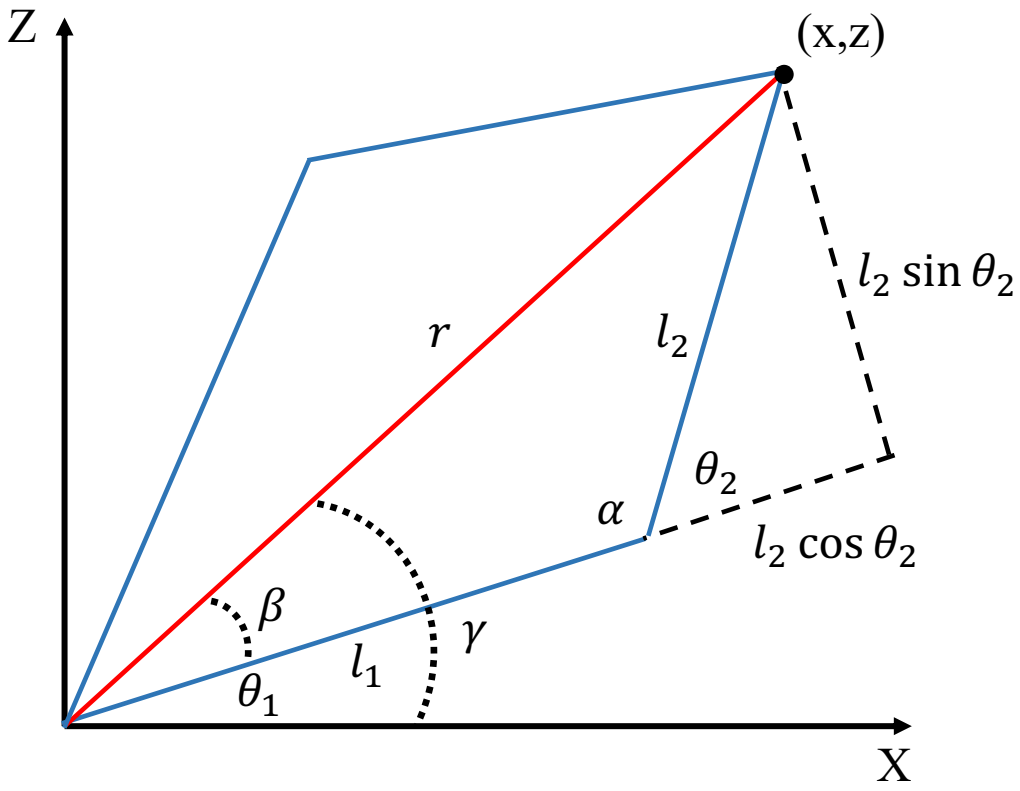
- New variables: γ, β

$$\gamma = \tan^{-1}\left(\frac{z}{x}\right) \quad \theta_1 = \gamma - \beta$$

- Therefore:

$$\theta_1 = \tan^{-1}\left(\frac{z}{x}\right) - \tan^{-1}\left(\frac{l_2 \sin\theta_2}{l_1 + l_2 \cos\theta_2}\right)$$

Analytical Inverse Kinematics

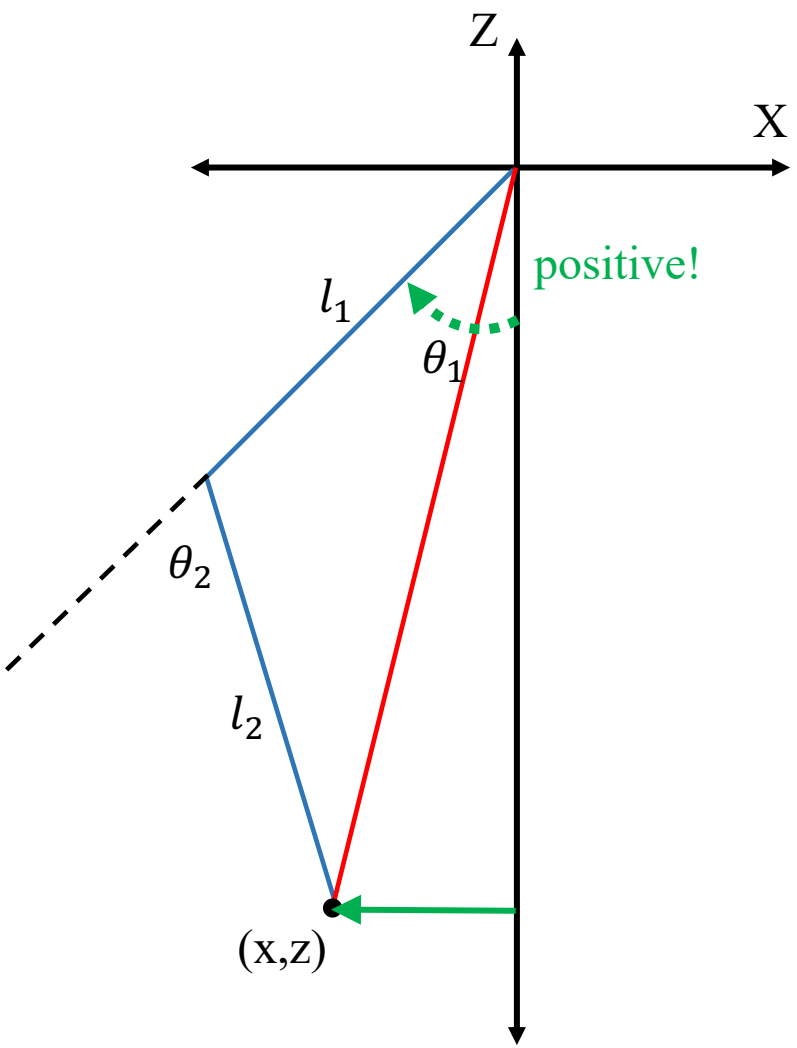


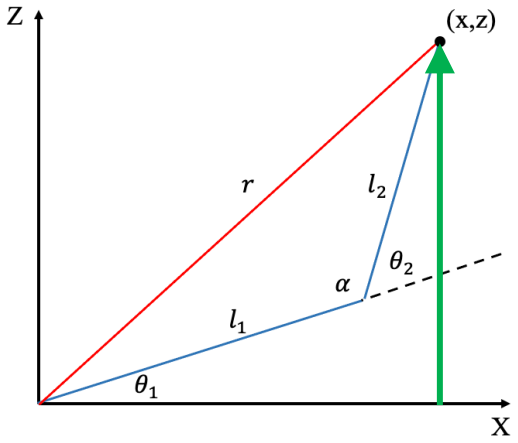
- Summary (both solutions):

$$\theta_2 = \pm\cos^{-1}\left(\frac{x^2 + z^2 - l_1^2 - l_2^2}{2l_1 l_2}\right)$$

$$\theta_1 = \tan^{-1}\left(\frac{z}{x}\right) \mp \tan^{-1}\left(\frac{l_2 \sin\theta_2}{l_1 + l_2 \cos\theta_2}\right)$$

Careful: Pybullet Reference Frame





$$\theta_2 = \pm\cos^{-1}\left(\frac{x^2+z^2-l_1^2-l_2^2}{2l_1l_2}\right)$$

$$\theta_1 = \tan^{-1}\left(\frac{z}{x}\right) \mp \tan^{-1}\left(\frac{l_2\sin\theta_2}{l_1+l_2\cos\theta_2}\right)$$

$$\theta_2 = \mp\cos^{-1}\left(\frac{x^2+z^2-l_1^2-l_2^2}{2l_1l_2}\right)$$

$$\theta_1 = np.arctan2(-x,-z) - \tan^{-1}\left(\frac{l_2\sin\theta_2}{l_1+l_2\cos\theta_2}\right)$$

Iterative Inverse Kinematics

- What if our system is more complex?

Algorithm 1: Iterative inverse kinematics

```
Data: p_des, q^0 ;                    /* Desired foot position, initial joint angle guess */
Result: q ;                                          /* Joint angles satisfying p=f(q)  */
q ← q^0;
p ← f(q^0);
while ||p − p_des|| > tol do
    J ← J(q) ;                                       /* Evaluate Jacobian for current q */
    J^+ ← (J)^+ ;                                          /* Update the pseudoinverse */
    Δp ← p_des − p ;                                     /* find the end-effector error */
    q ← q + αJ^+Δp ;                           /* update the joint angles (step size α) */
end
```

Iterative Inverse Kinematics

$$\boldsymbol{v} = \boldsymbol{J}(\boldsymbol{q})\dot{\boldsymbol{q}}$$

$$\dot{\boldsymbol{q}} = \boxed{\boldsymbol{J}^{-1}(\boldsymbol{q})}\boldsymbol{v}$$

Exists?

- What if our system is more complex?

Algorithm 1: Iterative inverse kinematics

Data: $\boldsymbol{p}_{\text{des}}, \boldsymbol{q}^0$; /* Desired foot position, initial joint angle guess */
Result: $\boldsymbol{q}$; /* Joint angles satisfying p=f(q) */
$\boldsymbol{q} \leftarrow \boldsymbol{q}^0$;
$\boldsymbol{p} \leftarrow f(\boldsymbol{q}^0)$;
while $\|\boldsymbol{p} - \boldsymbol{p}_{des}\| > tol$ **do**
  $\boldsymbol{J} \leftarrow \boldsymbol{J}(\boldsymbol{q})$; /* Evaluate Jacobian for current q */
  $\boldsymbol{J}^+ \leftarrow (\boldsymbol{J})^+$; /* Update the pseudoinverse */
  $\Delta\boldsymbol{p} \leftarrow \boldsymbol{p}_{\text{des}} - \boldsymbol{p}$; /* find the end-effector error */
  $\boxed{\boldsymbol{q} \leftarrow \boldsymbol{q} + \alpha\boldsymbol{J}^+\Delta\boldsymbol{p}}$; /* update the joint angles (step size α) */
end

Moore-Penrose Pseudo-Inverse

- Generalization of the matrix inversion operation for non-square matrices
- Consider a general matrix $A \in \mathbb{R}^{m \times n}$. When $m \geq n$ and $rank(A) = n$, define the *left pseudo-inverse* A_l^+, with $A_l^+ A = \mathbb{I}^{n \times n}$

$$A_l^+ \coloneqq (A^T A)^{-1} A^T$$

- If $m \leq n$ and $rank(A) = m$, define the *right pseudo-inverse* A_r^+, with $A A_r^+ = \mathbb{I}^{m \times m}$

$$A_r^+ \coloneqq A^T (A A^T)^{-1}$$

- To handle singularities, define the *damped pseudo-inverses* with damping factor λ as:

$$A_l^+ \coloneqq (A^T A + \lambda^2 \mathbf{I}_{n \times n})^{-1} A^T$$

$$A_r^+ \coloneqq A^T (A A^T + \lambda^2 \mathbf{I}_{m \times m})^{-1}$$

Implement the Inverse Kinematics Solvers

- Code at https://gitlab.epfl.ch/bellegar/lr-leg
- Upload your working code in Assignment 03 (1 per team)
- Today's assignment is not graded